

Python E-Course

Module 6 — Strings

Detailed Notes

Module Overview

Level: Beginner

Duration: ~1 week

Prerequisites: Modules 1-5

What You'll Learn

- Manipulate strings using indexing, slicing, and methods.
- Format text with f-strings and the format() mini-language.
- Use regular expressions for pattern matching.
- Understand encoding basics (UTF-8 vs bytes).

Lessons

#	Lesson	Duration
1	String Anatomy & Slicing	30 min
2	String Methods	40 min
3	Formatting Deep Dive	30 min
4	Regular Expressions	40 min
5	Encoding & Bytes	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 7: File Handling.

Lesson 1: String Anatomy & Slicing

Module: 6 — Strings

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Index and slice strings.
- Use len, in, and * on strings.
- Understand immutability.

Prerequisites

- Module 5, especially list slicing.

1. Why This Matters

Strings are the most-used data type after numbers. Logs, names, URLs, file paths, JSON keys, error messages — they're all strings. You already use them; this module makes you fluent.

2. Concept

2.1 A string is a sequence of characters

```
s = "Python"
print(s[0])    # 'P'
print(s[-1])   # 'n'
print(len(s))  # 6
```

Same indexing rules as lists. Strings are also immutable — `s[0] = "J"` raises `TypeError`.

2.2 Slicing

`s[start:stop:step]` returns a new string:

```
s = "Python"
s[0:3]      # 'Pyt'
s[2:]       # 'thon'
s[:3]       # 'Pyt'
s[::-1]     # 'nohtyP' (reversed)
s[::2]      # 'Pto'
```

2.3 Concatenation and repetition

```
"foo" + "bar"    # 'foobar'
"_" * 10         # '-----'
```

+ only between strings; convert with `str(x)` first if mixing types.

2.4 Membership

```
"py" in "python"    # True
"x" not in "python"  # True
```

2.5 Iteration

```
for ch in "hi":  
    print(ch)
```

2.6 Multi-line and raw strings (refresher)

```
multi = """line one  
line two"""  
path = r"C:\Users\Maya"
```

3. Code Examples

Example 1: Index and slice

```
s = "Programming"  
print(s[0])  
print(s[-1])  
print(s[3:7])  
print(s[::-1])
```

Expected Output:

```
P  
g  
gram  
gnimmargorP
```

Example 2: Concatenate and repeat

```
greeting = "Hi" + ", " + "Maya"  
divider = "*" * 12  
print(divider)  
print(greeting)  
print(divider)
```

Expected Output:

```
*****  
Hi, Maya  
*****
```

Example 3: Membership

```
email = "user@example.com"  
if "@" in email:  
    print("looks like an email")
```

Expected Output: looks like an email

Example 4: Immutability

```
s = "hello"  
try:  
    s[0] = "H"  
except TypeError as e:  
    print("Error:", e)  
print(s)
```

Expected Output:

```
Error: 'str' object does not support item assignment  
hello
```

Example 5: Iterate

```
for ch in "abc":  
    print(ch.upper())
```

Expected Output:

```
A  
B  
C
```

4. Common Mistakes

Mistake	What Happens	Fix
Trying to mutate <code>s[0] = ...</code>	<code>TypeError</code>	Build a new string
<code>"3" + 4</code>	<code>TypeError</code>	Convert: <code>"3" + str(4)</code>
Using <code>is</code> to compare strings	Sometimes <code>True</code> , unreliable	Use <code>==</code>
Index out of range	<code>IndexError</code>	Check with <code>len()</code> first

5. Practice Tasks

- Print the third character and last character of "hippopotamus".
- Reverse "abcdef" with slicing.
- Print every other letter of "alphabet".

6. Key Takeaways

- Strings: ordered, indexed, immutable sequences of characters.
- Slicing follows the same rules as lists.
- `+`, `*`, `in`, `len` all work.

7. What's Next

String methods — dozens of useful tools for cleaning, splitting, joining, and testing text.

Lesson 2: String Methods

Module: 6 — Strings

Duration: 40 minutes

Difficulty: Beginner

Learning Objectives

- Use the most common string methods to clean, split, join, and test text.
- Know which methods return a new string (most do — strings are immutable).
- Combine methods to handle real-world text problems.

Prerequisites

- Module 6 Lesson 1

1. Why This Matters

User input has trailing whitespace. CSV rows need splitting on commas. URLs need their https:// stripped. Almost every program manipulates text — knowing the right method beats writing 5-line loops every time.

2. Concept

2.1 Case methods

Method	Result
<code>.upper()</code>	All uppercase
<code>.lower()</code>	All lowercase
<code>.title()</code>	Title Case
<code>.capitalize()</code>	First letter upper, rest lower
<code>.swapcase()</code>	Flip case

```
"hello world".title()    # 'Hello World'
"PYTHON".lower()        # 'python'
```

2.2 Trimming whitespace

Method	Removes
<code>.strip()</code>	From both ends
<code>.lstrip()</code>	From left
<code>.rstrip()</code>	From right
<code>.strip("/x")</code>	Specified characters from ends

```
"  hi  ".strip()        # 'hi'
"###name###".strip("#")  # 'name'
```

2.3 Searching

Method	Returns
<code>.find(sub)</code>	Index, or -1 if not found
<code>.index(sub)</code>	Index, or raises ValueError

<code>.count(sub)</code>	Count of occurrences
<code>.startswith(s)</code>	Bool
<code>.endswith(s)</code>	Bool

```
"hello world".find("world")    # 6
"hello".startswith("he")      # True
```

2.4 Replacing

```
"banana".replace("a", "o")      # 'bonono'
"banana".replace("a", "o", 1)   # 'bonana' (only first)
```

2.5 Splitting and joining

```
"a,b,c".split(",")              # ['a', 'b', 'c']
"a b c".split()                 # ['a', 'b', 'c'] (any whitespace)
"-".join(["a", "b", "c"])       # 'a-b-c'
```

`.split()` without args splits on any run of whitespace — useful for sentences.

2.6 Testing content

Method	True when...
<code>.isdigit()</code>	All chars are digits
<code>.isalpha()</code>	All chars are letters
<code>.isalnum()</code>	All chars are letters or digits
<code>.isspace()</code>	All chars are whitespace
<code>.islower()</code> / <code>.isupper()</code>	All cased letters lowercase/uppercase

These return False on empty strings — don't rely on them for empty-string handling.

2.7 Splitting on lines

```
text = "line 1\nline 2\nline 3"
text.splitlines()              # ['line 1', 'line 2', 'line 3']
```

2.8 Immutability reminder

All methods return new strings. The original is unchanged.

```
s = "hello"
s.upper()
print(s)    # still 'hello'
s = s.upper()
print(s)    # 'HELLO'
```

3. Code Examples

Example 1: Cleaning input

```
raw = "  Hello, World!  \n"
cleaned = raw.strip()
print(repr(cleaned))
print(cleaned.lower())
```

Expected Output:

```
'Hello, World!'
hello, world!
```

Example 2: Split and join

```
csv = "apple,banana,cherry"
items = csv.split(",")
print(items)
joined = " | ".join(items)
print(joined)
```

Expected Output:

```
['apple', 'banana', 'cherry']
apple | banana | cherry
```

Example 3: Replace

```
sentence = "I love cats. Cats are great."
print(sentence.replace("cats", "dogs").replace("Cats", "Dogs"))
```

Expected Output: I love dogs. Dogs are great.

Example 4: Validate input

```
user = " maya123 "
clean = user.strip()
if clean.isalnum():
    print("Valid username:", clean)
else:
    print("Bad username:", clean)
```

Expected Output: Valid username: maya123

Example 5: Counting and finding

```
text = "to be or not to be"
print(text.count("be"))
print(text.find("not"))
print(text.startswith("to"))
```

Expected Output:

```
2
9
True
```

Example 6: Chaining

```
raw = " HELLO, World! "
print(raw.strip().lower().replace("hello", "hi"))
```

Expected Output: hi, world!

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting to assign result	String unchanged	s = s.lower(), not s.lower()

s.find returns -1, treated as truthy index	Bug	Use if s.find(x) != -1: or if x in s:
.split(" ") vs .split()	Empty strings appear with " " on double-spaces	.split() with no args handles any whitespace
isdigit on empty string	False, may surprise you	Check len > 0 separately if needed

5. Practice Tasks

- Take " Python is FUN " and produce "python is fun" using strip + lower.
- Split "name=Maya;age=22;city=Pune" on ;, then = to build a dict.
- Count vowels in "computer science" using a loop and in "aeiou".

6. Key Takeaways

- Methods return new strings; the original is immutable.
- strip/split/join/replace cover 80% of text manipulation.
- find returns -1 for missing; index raises.
- Chain methods for compact, readable transformations.

7. What's Next

A deeper look at string formatting with f-strings and the format mini-language.

Lesson 3: Formatting Deep Dive

Module: 6 — Strings

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Use f-string format specifiers for width, alignment, precision, padding, and bases.
- Use the = self-documenting f-string for debugging.
- Pick between f-strings, str.format, and %.

Prerequisites

- Module 2 Lesson 6

1. Why This Matters

Formatting is the difference between an ugly script and a polished tool. Aligned tables, currency, percentages, debug output — f-strings handle all of it once you know the mini-language.

2. Concept

2.1 The format spec

After a : inside {}:

```
{value:[fill][align][sign][#][0][width][,][.precision][type]}
```

You won't use all of these together; the common ones:

Spec	Meaning	Example	Output
:10	Width 10 (left-aligned default for strings)	<code>f'{'hi':10}</code>	<code>"</code>
:<10	Left align width 10	<code>f'{'hi':<10}</code>	<code>"</code>
:>10	Right align width 10	<code>f'{'hi':>10}</code>	<code>"</code>
:^10	Center width 10	<code>f'{'hi':^10}</code>	<code>"</code>
:*^10	Center, fill with *	<code>f'{'hi':*^10}"</code>	<code>'*hi*'</code>
:.3f	Float, 3 decimals	<code>f'{3.14159:.3f}"</code>	<code>'3.142'</code>
:,.2f	Thousands, 2 decimals	<code>f'{1234567.5:,.2f}"</code>	<code>'1,234,567.50'</code>
:.0%	Percent, 0 decimals	<code>f'{0.853:.0%}"</code>	<code>'85%'</code>
:b,:o,:x	Binary, octal, hex	<code>f'{255:x}"</code>	<code>'ff'</code>
:+	Always show sign	<code>f'{5:+}"</code>	<code>'+5'</code>
:05d	Pad int with zeros, width 5	<code>f'{42:05d}"</code>	<code>'00042'</code>

2.2 Self-documenting = (Python 3.8+)

Inside an f-string, append = to print both the expression and its value — perfect for quick debugging:

```
x = 42
print(f"{x=}")          # 'x=42'
print(f"{x*2=}")        # 'x*2=84'
```

2.3 Calling expressions inside {}

You can use any expression:

```
name = "Maya"
print(f"Length: {len(name)}")
print(f"Upper: {name.upper()}")
print(f"Sum: {2 + 3}")
```

2.4 Old-style — when you'll see it

```
"%s is %d" % ("Maya", 22)          # %-formatting (oldest)
"{ } is {}".format("Maya", 22)     # str.format
"{name} is {age}".format(name="Maya", age=22)
```

You don't need to write these but you'll read them in older code.

3. Code Examples

Example 1: Aligned table

```
items = [("Apple", 50, 3), ("Bread", 40, 1), ("Milk", 60, 2)]
print(f"{'Item':<10}{'Price':>8}{'Qty':>6}{'Total':>10}")
print("-" * 34)
for name, price, qty in items:
    print(f"{'name':<10}{'price':>8.2f}{'qty':>6}{'price*qty':>10.2f}")
```

Expected Output:

Item	Price	Qty	Total
Apple	50.00	3	150.00
Bread	40.00	1	40.00
Milk	60.00	2	120.00

Example 2: Currency and percent

```
revenue = 2_345_678
margin = 0.214
print(f"Revenue: ₹{revenue:,.0f}")
print(f"Margin: {margin:.1%}")
```

Expected Output:

```
Revenue: ₹2,345,678
Margin: 21.4%
```

Example 3: Self-documenting debug

```
a = 7
b = 3
print(f"{a=}, {b=}, {a/b:.2f}")
```

Expected Output: a=7, b=3, a/b=2.33

Example 4: Base conversions

```
n = 255
print(f"dec={n} hex={n:x} oct={n:o} bin={n:b}")
```

Expected Output: dec=255 hex=ff oct=377 bin=11111111

Example 5: Padding

```
for i in [3, 21, 105]:
    print(f"ID-{i:06d}")
```

Expected Output:

```
ID-000003
ID-000021
ID-000105
```

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting the f	Braces show literally	Add f
:.2f on int	Works (5:.2f → '5.00') but be intentional	Convert beforehand if needed
:.2f on string	ValueError	Convert with float() first
Wrong align character	:?10 raises	Use <, >, or ^

5. Practice Tasks

- Print numbers 1-10 right-aligned in width 4.
- Print a price 12345.6789 with thousands separator and 2 decimals.
- Use f"{x=}" debug syntax to print three variables of your choice.

6. Key Takeaways

- f-strings + format spec cover every formatting need.
- :width, :align, :Nf, :, :%, :ONd, :x are the workhorses.
- f"{var=}" is invaluable for debugging.

7. What's Next

Regular expressions — patterns that match across strings.

Lesson 4: Regular Expressions

Module: 6 — Strings

Duration: 40 minutes

Difficulty: Beginner

Learning Objectives

- Use the re module to search and match text.
- Read basic regex patterns: literals, character classes, quantifiers, anchors.
- Extract matches with groups.
- Replace text with re.sub.

Prerequisites

- Module 6 Lesson 2

1. Why This Matters

Some text problems — finding all phone numbers in a document, parsing log lines, validating emails — get ugly fast with string methods. Regex is a small, focused language for describing patterns. It's intimidating at first; once it clicks, it's indispensable.

2. Concept

2.1 Importing

```
import re
```

2.2 Common functions

Function	Use
re.search(pat, text)	Find first match anywhere
re.match(pat, text)	Match at start only
re.fullmatch(pat, text)	Match the entire string
re.findall(pat, text)	All matches as a list
re.finditer(pat, text)	All matches as Match objects
re.sub(pat, repl, text)	Replace matches
re.split(pat, text)	Split by pattern

2.3 Pattern building blocks

Pattern	Matches
abc	Literal abc
.	Any single char (not newline)
\d	Digit [0-9]
\D	Non-digit
\w	Word char [A-Za-z0-9_]
\W	Non-word
\s	Whitespace

\S	Non-whitespace
[abc]	Any of a, b, c
[^abc]	Not a/b/c
[a-z]	Range
^	Start of string
\$	End of string

2.4 Quantifiers

Quantifier	Meaning
*	0 or more
+	1 or more
?	0 or 1
{n}	Exactly n
{n,m}	Between n and m

2.5 Groups

(...) captures a group. Access via .group(N) on the match object.

```
m = re.search(r"(\d+)-(\d+)", "id 42-99 ok")
m.group(0)    # '42-99' (whole match)
m.group(1)    # '42'
m.group(2)    # '99'
```

2.6 Raw strings

Always use raw strings (r"...") for patterns — otherwise backslashes get tangled with Python's own escapes.

2.7 Match vs None

search, match, fullmatch return a Match object or None. Check before accessing.

```
m = re.search(r"\d+", "no number")
if m:
    print(m.group())
```

3. Code Examples

Example 1: Search for digits

```
import re
m = re.search(r"\d+", "order 1234 placed")
print(m.group())
```

Expected Output: 1234

Example 2: Find all phone-like patterns

```
import re
text = "Call 555-1234 or 555-5678 today."
print(re.findall(r"\d{3}-\d{4}", text))
```

Expected Output: ['555-1234', '555-5678']

Example 3: Capturing groups

```
import re
m = re.search(r"(\w+)@(\w+\.\w+)", "Email: maya@example.com")
if m:
    print("user:", m.group(1))
    print("host:", m.group(2))
```

Expected Output:

```
user: maya
host: example.com
```

Example 4: Replace

```
import re
text = "Phone: 555-1234, Fax: 555-9876"
masked = re.sub(r"\d", "*", text)
print(masked)
```

Expected Output: Phone: -, Fax: -*****

Example 5: Validate

```
import re
def looks_like_zip(code):
    return re.fullmatch(r"\d{5}", code) is not None

print(looks_like_zip("12345"))
print(looks_like_zip("123"))
print(looks_like_zip("12345a"))
```

Expected Output:

```
True
False
False
```

Example 6: Split on whitespace runs

```
import re
text = "one two\tthree\nfour"
print(re.split(r"\s+", text))
```

Expected Output: ['one', 'two', 'three', 'four']

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting r"..."	Backslash chaos	Always use raw strings
Calling .group() on None	AttributeError	Check if m: first
re.match thinking it scans the whole string	Only anchors at start	Use re.search for anywhere
Confusing (any number) with literal	Pattern misbehaves	Escape: *

5. Practice Tasks

- Extract all numbers from "abc123def456ghi" using `re.findall`.
- Validate an email-ish string with `re.fullmatch(r"\w+@\w+\.\w+", ...)`.
- Replace all whitespace runs in a sentence with a single underscore.

6. Key Takeaways

- `re` is the standard library regex module.
- Use raw strings for patterns.
- Common functions: `search`, `findall`, `sub`, `fullmatch`, `split`.
- Quantifiers + character classes + groups cover most needs.

7. What's Next

The last lesson — encoding and bytes — important for file and network work coming in Modules 7 and 12.

Lesson 5: Encoding & Bytes

Module: 6 — Strings

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Distinguish str (text) from bytes (raw bytes).
- Encode to bytes and decode back, especially with UTF-8.
- Know why character encoding matters for files and APIs.

Prerequisites

- Module 6 Lesson 4

1. Why This Matters

Files and network protocols carry bytes — not characters. Without understanding encoding, you'll hit `UnicodeDecodeError` the moment a non-ASCII character shows up in your data. Five minutes of theory here saves hours later.

2. Concept

2.1 str is Unicode

In Python 3, every string is a sequence of Unicode code points — abstract identifiers for characters across every script (Latin, Hindi, Chinese, emoji...).

```
s = "café – naïve résumé"
print(len(s))    # 19 – characters, not bytes
```

2.2 bytes is raw bytes

```
b = b"hello"
print(type(b))   # <class 'bytes'>
```

The `b""` prefix makes a bytes literal. Each element is an integer 0-255.

2.3 Encoding: str → bytes

```
s = "café"
b = s.encode("utf-8")
print(b)           # b'caf\xc3\xa9'
print(len(b))      # 5 – not 4!
```

UTF-8 uses 1-4 bytes per character. ASCII chars (a-z, digits) take 1 byte; many others take 2-4.

2.4 Decoding: bytes → str

```
b = b'caf\xc3\xa9'
s = b.decode("utf-8")
print(s)           # 'café'
```

If you decode with the wrong encoding, you get garbage or an error.

2.5 UTF-8 is the right default

- Web standards default to UTF-8.
- Python source files default to UTF-8.
- Most files you read should be UTF-8 (covered next module).

Other encodings exist (latin-1, utf-16, cp1252) for legacy data. Specify them explicitly when needed.

2.6 Errors and strict/replace

```
b = b'\xff'
b.decode("utf-8")           # UnicodeDecodeError
b.decode("utf-8", "replace") # '�' (replacement char)
b.decode("utf-8", "ignore")  # ''
```

3. Code Examples

Example 1: Encode and decode

```
s = "naïve"
b = s.encode("utf-8")
print(b)
print(len(s), len(b))
print(b.decode("utf-8"))
```

Expected Output:

```
b'na\xc3\xafve'
5 6
naïve
```

Explanation: 5 Unicode chars become 6 bytes — ï takes 2.

Example 2: ASCII vs UTF-8

```
print("hi".encode("utf-8"))    # b'hi'
print("résumé".encode("utf-8")) # uses multi-byte chars
```

Expected Output:

```
b'hi'
b'r\xc3\xa9sum\xc3\xa9'
```

Example 3: Hex view

```
s = "😄"
print(s.encode("utf-8"))
```

Expected Output: b'\xf0\x9f\x98\x80'

Explanation: Emojis take 4 bytes in UTF-8.

Example 4: Wrong decoding

```
b = "Olá".encode("utf-8")
try:
    print(b.decode("ascii"))
except UnicodeDecodeError as e:
    print("Error:", e)
```

Expected Output: Error: 'ascii' codec can't decode byte 0xc3 in position 1: ordinal not in range(128)

Example 5: Replacement on bad bytes

```
b = b"\xff\xfe hello"
print(b.decode("utf-8", "replace"))
```

Expected Output:   hello

4. Common Mistakes

Mistake	What Happens	Fix
Reading file without specifying encoding	OS-dependent default — bugs on Windows	<code>open(..., encoding="utf-8")</code>
Mixing str and bytes with +	<code>TypeError</code>	Encode/decode first
Assuming <code>len()</code> is bytes	It's chars	Use <code>len(s.encode("utf-8"))</code> for bytes
Storing emojis in latin-1	Mojibake	Always UTF-8

5. Practice Tasks

- Encode "hello" and your own name to UTF-8 and print the bytes and their length.
- Decode `b'caf\xc3\xa9'` back to 'café'.
- Try `"€".encode("utf-8")` — note how many bytes the Euro sign takes.

6. Key Takeaways

- str is Unicode; bytes is raw bytes.
- `.encode("utf-8")` and `.decode("utf-8")` round-trip safely.
- Default to UTF-8 everywhere.
- Use `errors="replace"` only when you accept losing data.

7. What's Next

End of Module 6. Next: Module 7 — File Handling — read and write actual files on disk.

Module 6 — Exercises

15 exercises (3 per lesson).

6.1 Anatomy

- (E) Print the 3rd and last characters of "interaction".
- (M) Reverse "Python" with slicing.
- (H) Print every 3rd character of "abcdefghij".

6.2 Methods

- (E) Strip whitespace from " hello " and print.
- (M) Split "red,green,blue;yellow,purple" first on ; then on , to get a flat list.
- (H) Count vowels in "the quick brown fox".

6.3 Formatting

- (E) Format 3.14159 to 2 decimals.
- (M) Print numbers 1-5 right-aligned in width 4.
- (H) Build a 3-column table of items (name, price, qty) for at least 3 items.

6.4 Regex

- (E) Find all digits in "abc123xyz456".
- (M) Validate "abc-1234" matches `\w{3}-\d{4}`.
- (H) Replace all sequences of 1+ spaces with a single dash in a sentence of your choice.

6.5 Encoding

- (E) Encode "hello" to UTF-8 and print.
- (M) Decode `b'\xe4\xb8\xad'` from UTF-8 (it's a Chinese character).
- (H) Encode an emoji to UTF-8 and print the byte count.

Module 6 — Quiz

10 MCQs.

Q1

What does `"abcdef"[1:4]` return? A. `"bcd"` B. `"abc"` C. `"bcde"` D. `"abcd"`

Answer: A — start inclusive, stop exclusive. (L1)

Q2

What does `" hi ".strip()` return? A. `"hi "` B. `" hi"` C. `"hi"` D. `" hi "`

Answer: C (L2)

Q3

What does `"a,b,c".split(",")` produce? A. `("a","b","c")` B. `["a","b","c"]` C. `"abc"` D. error

Answer: B (L2)

Q4

What does `"-".join(["a","b","c"])` produce? A. `"a-b-c"` B. `"-abc"` C. `["a-","b-","c"]` D. error

Answer: A (L2)

Q5

What does `f"{3.14:.1f}"` produce? A. `"3.1"` B. `"3.14"` C. `"3"` D. `"3.10"`

Answer: A (L3)

Q6

Which f-string spec right-aligns in width 8? A. `:>8` B. `:<8` C. `:^8` D. `:|8`

Answer: A (L3)

Q7

Which regex matches one or more digits? A. `\d*` B. `\d+` C. `\d?` D. `\d`

Answer: B (L4)

Q8

What does `re.findall(r"\d+", "a1b22c333")` return? A. `["1","2","3"]` B. `["1","22","333"]` C. `"1 22 333"` D. `[]`

Answer: B (L4)

Q9

What's the right way to encode a string to bytes in UTF-8? A. `s.encode("utf-8")` B. `bytes(s)` C. `s.to_bytes()` D. `utf8(s)`

Answer: A (L5)

Q10

What's the result of `len("café")` in Python 3? A. 4 B. 5 C. depends on encoding D. error

Answer: A — `len` counts Unicode characters. (L5)

Module 6 — Assignment: Text Statistics

Estimated time: 1.5 hours

Combines: Module 6 + Modules 4-5

Brief

Build a text-analysis tool that reports statistics on a paragraph: characters, words, sentences, most common words, average word length, and longest word.

Requirements

Implement functions:

- `char_count(text)` — total chars (incl. spaces).
- `word_count(text)` — total words (split on whitespace).
- `sentence_count(text)` — count of ., !, ? (use regex).
- `avg_word_length(text)` → float, rounded 2 decimals.
- `longest_word(text)` → the longest word (ties broken by first occurrence).
- `top_words(text, n=3)` → list of (word, count) for the top n most-common words (case-insensitive, ignore punctuation).
- `report(text)` — calls all the above and prints a formatted report.

Then call `report(sample)` with a hard-coded sample paragraph at the bottom.

Constraints

- Modules 1-6 only.
- Use string methods + dict for counts + at most one re use for sentence counting.
- f-strings for output.

Expected Behavior (sample)

```
== Text Report ==
Characters      : 132
Words           : 24
Sentences       : 3
Avg word len    : 4.50
Longest word    : intelligence
Top 3 words     :
  1. the (3)
  2. and (2)
  3. data (2)
```

Grading Rubric

Criterion	Weight
All 7 functions work	40%
Top-N is case-insensitive and strips punctuation	15%

Uses dict + sort with key=	15%
Report layout	15%
Style + docstrings	15%

Submission

Save as assignment_module6.py.

Module 6 — Mini Project: Smart Form Validator

Overview

A small command-line "form" that prompts the user for name, email, phone, and password, then validates each field with string methods and regex. Re-prompts on bad input until valid.

Concepts Used

- String methods (M6 L2)
- f-strings (M6 L3)
- Regex (M6 L4)
- Functions, loops, conditionals (M3-4)

Features

- Name: 2-50 chars, letters and spaces only.
- Email: matches `\w+@\w+\.\w+`.
- Phone: 10 digits, may have separators (-, space).
- Password: 8+ chars, ≥ 1 letter, ≥ 1 digit.

For each field, keep prompting until valid; show a clear error message on failure.

Starter Code

```
# Mini Project: Smart Form Validator
import re

def valid_name(s):
    """At least 2, at most 50 chars; only letters and spaces."""
    s = s.strip()
    return 2 <= len(s) <= 50 and all(c.isalpha() or c.isspace() for c in s)

def valid_email(s):
    return re.fullmatch(r"\w+@\w+\.\w+", s.strip()) is not None

def valid_phone(s):
    digits = re.sub(r"[s-]", "", s)
    return digits.isdigit() and len(digits) == 10

def valid_password(s):
    if len(s) < 8:
        return False
    has_letter = any(c.isalpha() for c in s)
    has_digit = any(c.isdigit() for c in s)
    return has_letter and has_digit

def prompt_until_valid(label, validator, error_msg):
    while True:
        value = input(f"{label}: ")
        if validator(value):
            return value.strip()
```

```

        print(f"  ✗ {error_msg}")

def run():
    print("== Sign up ==")
    name = prompt_until_valid("Name", valid_name,
                              "2-50 letters/spaces only")
    email = prompt_until_valid("Email", valid_email,
                              "Must look like user@host.tld")
    phone = prompt_until_valid("Phone", valid_phone,
                              "Need 10 digits (separators allowed)")
    password = prompt_until_valid("Password", valid_password,
                                  "8+ chars, must have at least one letter and one digit")
    print("\nAccount created!")
    print(f>Welcome, {name}.")

if __name__ == "__main__":
    run()

```

Expected Interaction

```

== Sign up ==
Name: A
  ✗ 2-50 letters/spaces only
Name: Maya Sharma
Email: maya@
  ✗ Must look like user@host.tld
Email: maya@x.com
Phone: 555 1234
  ✗ Need 10 digits (separators allowed)
Phone: 555-123-4567
Password: short
  ✗ 8+ chars, must have at least one letter and one digit
Password: hello123
Account created!
Welcome, Maya Sharma.

```

Extension Challenges

- Add a "strong password" check: require at least one symbol.
- Accept Indian phone numbers (10 digits starting with 6-9).
- Mask password input (use `getpass.getpass` — preview of M10).

Grading Rubric

Criterion	Weight
All 4 validators correct	40%
<code>prompt_until_valid</code> reused	20%
Clear, friendly error msgs	15%
Regex used appropriately	15%
Style + naming	10%